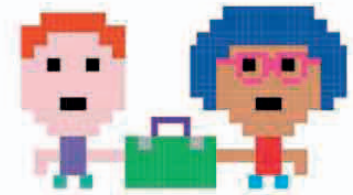


Passing data to functions

A function has to be told which values to work with. For example, in `print(a, b, c)`, the function `print()` is being passed the values `"a"`, `"b"`, and `"c"`. In `height(1, 45)`, the values 1 and 45 are being passed to the function `height`.



1 Add parameters to the function

Values passed to a function are called "parameters". Parameters are put inside the parentheses next to the function's name in its definition.

```
def height(m, cm):
    total = (100 * m) + cm
    print(total, 'cm tall')
```

"m" and "cm" are the parameters

Prints the value of "total" followed by "cm tall"

To work out the total in "cm", the value of "m" needs to be multiplied by 100 (because 1 m = 100 cm)

2 Values are defined

The code inside the function uses the values that are passed to it.

```
>>> height(1, 45)
145 cm tall
```

Calls the function to give the answer when "m" = 1 and "cm" = 45

Shows that 1 m 45 cm is equal to 145 cm

Getting data back from functions

Functions are most useful when they send some data back to the program—a return value. To make a function return a value, add `"return"` followed by the value to be sent back.



1 Define a function that returns a number

Python's `input()` function always returns a string, even if a number is entered. The new function below gives back a number instead.

```
def num_input(prompt):
    typed = input(prompt)
    num = int(typed)
    return num
```

The number is stored as a string in the variable "typed"

Returns the value stored in the variable

This converts the string into a number and stores it in the variable "num"

```
a = num_input('Enter a')
b = num_input('Enter b')
print('a + b =', a + b)
```

2 Number as output

If the program used the function `input`, `"a + b"` would put the strings `"10"` and `"7"` together to give `"107"`.

```
Enter a 10
Enter b 7
a + b = 17
```

Adding `"a + b"` outputs `"17"` because the function `"num_input"` gives back numbers, not strings